

Assignment Title	Smart Solar Microgrid Trading System – Client-Server Application (Web, Mobile and Web Service)
Learning outcomes covered	LO 2: Apply cutting-edge theory and skills in the design and implementation of Enterprise-Scale systems. LO 3: Assess the student with a thorough understanding of the tools and techniques that use to develop enterprise systems.
Assignment Mode	Group assignment – four (4) members per group
Maximum Marks	Source code and demo/viva: 95 marks Project report: 5 marks
Contribution to the Final Grade	20% of the module final mark (Project report 5% + source code and demo/viva 15%)
Date published	24/08/2026
Deadline for submissions	30th of September 2026, 11.59 PM
Mode of Submission	A single zipped file containing all project directories/files, the detailed report, and a screenshot of the main opening menu/screen of the app. The zip file name must include the IT number (e.g. IT15895623.zip). Viva is held as a supervised session after submission; attendance is compulsory.

Assignment description

This continuous assessment (project) is a group project, and in each group, there should be four members.

Project Specification

The purpose of this assignment is to create an end-to-end Smart Solar Microgrid Trading System using client-server architecture. The system features a **web application for back-office administration** and **microgrid site operators**, alongside a **mobile application for solar prosumers** (property owners with solar panel arrays).

a) Web Application:

- **User Management:** Create **web application users** with two distinct roles: **Backoffice** and **Grid Operator**. Only **Backoffice** users have access to **system administration functions**, while **Grid Operators** access **operational tools**.

- **Prosumer Management:** Create, update, and deactivate prosumer profiles using National Identity Card (NIC) as the primary key. Deactivated accounts can only be reactivated by a Backoffice officer.
- **Microgrid Node Management:** Manage solar grid hubs, including creating new hubs with GPS location, capacity specs (kW/h), and available battery storage slots. Update schedules and deactivate nodes (deactivation is blocked if active energy reservations exist).
- **Energy Slot Reservation Management:** Create, update, and cancel power trading reservations (must be scheduled within 7 days; updates and cancellations require at least 12 hours' notice).
- **User Interface:** Developed using Bootstrap 5, Tailwind CSS, or React.js for a responsive appearance.

b) Mobile Application: Mobile apps must be pure Android with SQLite DB (no frameworks)

- Mobile applications are intended to provide functionalities for both Solar Prosumer and grid operators.
 - **Native Specs:** Pure native Android with local SQLite database for local user management (no cross-platform frameworks allowed).
 - **Prosumer Account Control:** Prosumers register using NIC as the primary key, edit profile data, and request account deactivation.
 - **Reservation & QR Dispatch:** Prosumers reserve, modify, or cancel energy drop-off/charging slots. Once approved, the app generates a secure transaction QR code.
 - **Dashboard & Maps:** Display active/pending reservation counts, Bookings, and nearby grid nodes via Google Maps API. Additionally, user should be able to view booking history, pending booking and search booking
 - **Operator Mode:** Grid Operators log into the mobile app, scan the prosumer's transaction QR code, verify server data, and finalize energy transfer business logic.

c) Web Service

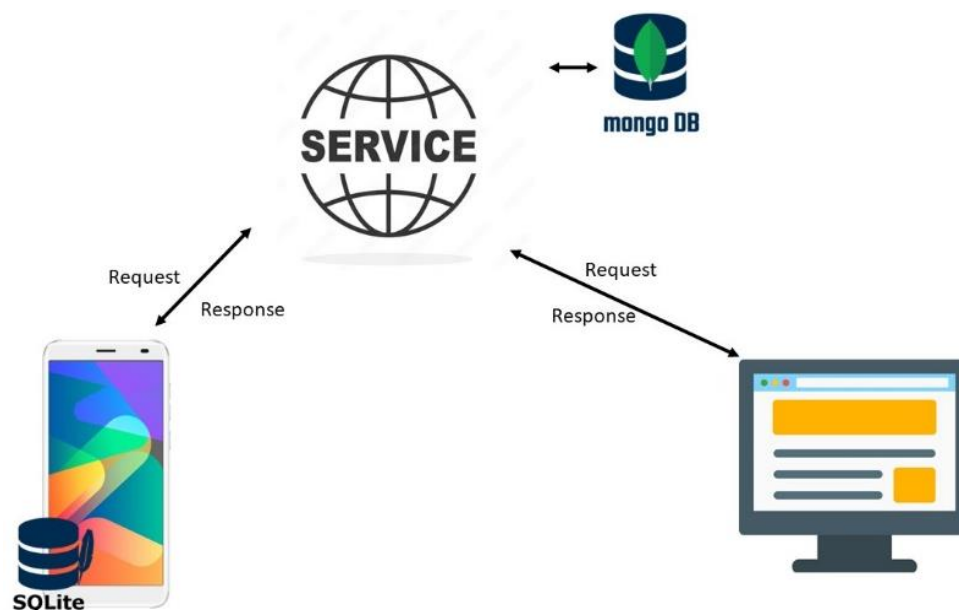
- **Architecture:** FAT Service pattern where all business logic resides strictly in the central API.
- **Tech Stack:** C# Web API deployed on Windows IIS Server with a NoSQL server-side database (e.g., MongoDB).

- **Client Constraints:** Both Web and Pure Android clients act as UI interfaces, maintaining local persistence where specified (SQLite on Android) and communicating exclusively via RESTful API calls to the central service.

The **web application** and **mobile application** should be limited to **user interface functionality** with Database and interact exclusively with the web service.

This system incorporates **web and mobile applications that connect to a centralized web service running on IIS**. Both client applications interact with the web service to access processed data and business logic. The server-side data storage utilizes a NoSQL database.

Therefore, it should develop an Android mobile application (Pure android and No frameworks) and web applications which have only interfaces to interact with Web service to facilitate data and Business logic and the NoSQL database. Here is the high-level diagram.



Project Scenario:

In this project scenario, the Backoffice team is responsible for managing the registration of solar microgrid nodes and maintaining their operational schedules. Grid operators, who can access both the web application and the mobile application, are responsible for updating battery slot availability and monitoring power trading bookings. Solar prosumers, on the other hand, can conveniently use the mobile application to reserve energy slots, modify existing reservations, and review their energy transfer history. Once a booking is confirmed, the prosumer receives a confirmation and can access the details directly on their dashboard. If circumstances

change, cancellations can be easily made either through the mobile application or with the assistance of a grid operator

What do you have to hand in?

1. Project Code AND a screenshot: Provide a single zipped file containing all directories/files in your project and a detailed report. You should also include a screen shot of the main opening menu/screen of your app. Include IT number in the zipped file name e.g. IT15895623.zip

Note: Code that does not include the following will not be marked

- **comment header block on each .cs file.**
 - **Inline comments at the beginning of each method.**
 - **Unique screenshots of the application.**
2. You should write a detail report that describes your app with following Criteria.
 - Screen shots of all UI's
 - Application high level diagram, Use case Diagram and DFD diagram.
 - Database design
 - Source code (**not screen shots**). Copy and paste your codes.
 - All the references
 - Git Repository link
 - Individual contribution
 - Challenges
 3. Create a README file containing the following:
 - Git repository link (individual contributions must be clearly mentioned)
 - Create a video of no more than 5 minutes explaining how the application works and provide a video link (Youtube or oneDrive) with README file.

Marks Allocation

This group project assignment is worth **20%** of your overall final mark in this module. Mark distribution is as follows.

Note: No exemptions are approved. It should clearly mention the contribution of each student.

Project report	5%
source code, and demo/viva (participation required)	15%

Due Date

Please submit the project source code and report on or before **30th of September 2026 11.59PM.**

What is Viva for the remaining assessment of the app?

Viva session will test your knowledge in your own app. It will be done as a supervised test (After submission).

If you are absent for the viva session, your assignment will not be marked.

Instructions

- All students are expected to submit assignments within the given deadline.
- late submissions are not accepted.
- Viva is treated as formal exam. No additional dates will be provided. You will get the date of viva at least a week before.
- Any code snippets that are not directly written by you (e.g., used from a tutorial) must be referenced as such within your code. You must directly comment the code to explain its source. **Failure to reference code that is not yours will be treated as plagiarism.**
- The app must be your own work. **Any plagiarism found will get zero marks**, and the plagiarism escalated as per DIT assessment regulations.

Usage of AI:

This assessment has been designed using the CLEAR Framework and is levelled against the AI Assessment Scale. It is set at **Level 2 — AI Planning**. AI tools — chat assistants, IDE copilots and agentic coding tools — may be used only during the initial planning stage of the project: brainstorming ideas, outlining the solution, and carrying out preliminary research into the domain and the technologies involved. Anything an AI tool produces at that stage must be critically evaluated, refined and developed further by you; it is a starting point, not an answer. The actual development and implementation of the system, including the C# Web API, the native Android application, the web application and the MongoDB database, must be completed independently without AI assistance. The submitted project must reflect your own understanding, technical skills, design decisions and development effort.

Evidence of how the work was produced forms part of the assessment. You are expected to develop under version control (GitHub) with meaningful, descriptive commits, and to document your design and development decisions in the report. In your individual contribution section of the report, disclose any AI tools you used during the planning stage and include a short reflection addressing. No marks are awarded for the disclosure or the reflection itself. At the viva, however, you will be asked structured questions about your submitted work, your development decisions and your use of AI, and you must be able to explain and justify how your initial ideas became the final solution. Any code, diagram or documentation you cannot explain or modify may receive reduced or zero individual marks.

Marking Scheme

The marks below follow the SE4040 marking scheme for this assignment. Table 1 covers the deliverables produced and submitted by the group. Table 2 covers what each member is assessed on individually.

Smart Solar Microgrid Trading System — Total 100 marks | Group contribution 35 | Individual contribution 65

Smart Solar Microgrid Trading System — Total 100 marks | Group contribution 35 | Individual contribution 65

Table 1 – Group Contribution (35 marks)

Criterion	Excellent	Good	Satisfactory	Poor	Very Poor
Service Architecture and API Design (8) Hosting the Web API / Service on IIS 4 · MongoDB connection 4	7 – 8 C# Web API hosted on IIS and reachable by both clients. Stable MongoDB connection used by all endpoints. FAT-service pattern followed — all business logic sits in the API.	5 – 6 API hosted and reachable. MongoDB connection works for most endpoints. Business logic mostly in the service layer.	4 API runs but hosting is partial or unstable. MongoDB used for a limited set of operations. Some logic has leaked into the clients.	2 – 3 API barely functional or not properly hosted. Unreliable database connection. Business logic mainly in the clients.	0 – 1 No working web service, or no MongoDB connection.
Database Design and Data Modelling (4) User's detail 1 · SolarStationInfo 1 · EnergyBookingSlots 1 · Energy Reservation 1	4 All four collections created with every required field. Sample data added manually or through the application. References between collections consistent.	3 All four collections present with most required fields. Adequate sample data.	2 Three collections present. Several required fields missing. Limited sample data.	1 Only one or two collections. Structure largely incorrect. Almost no data.	0 No usable MongoDB structure.
Client Application Build and Architecture Compliance (12) Pure native Android with SQLite 6 · Web application as a UI layer 6	10 – 12 Mobile app is pure native Android with no cross-platform framework and uses SQLite for local persistence. Web app acts purely as a UI layer. Both connect reliably to the hosted service and handle responses and errors gracefully.	8 – 9 Both clients built to specification and connect for most functions. Minor error-handling gaps.	6 – 7 Both clients run and each performs at least one function through the service. Some data handled outside the service.	3 – 5 One client barely functional. Connection unreliable. Significant business logic held in the clients.	0 – 2 No working client applications, or a prohibited cross-platform framework used.
User Interface and Experience Design (6) Mobile interfaces 2 · Web interfaces with Tailwind CSS / Bootstrap 5 2 · Home (index) page 1 · Completeness of all pages 1	5 – 6 Both Android and web interfaces are polished, consistent and responsive. Tailwind CSS or Bootstrap 5 applied throughout. Home page well designed. Every page implemented and working.	4 Attractive, consistent interfaces on most screens. Styling framework used correctly. Home page clear. Nearly all pages working.	3 Interfaces functional but plain or inconsistent. Framework used only in parts. Some pages incomplete.	2 Few designed screens. Little use of the framework. Cluttered or unresponsive layout. Several pages missing.	0 – 1 Interfaces largely absent or non-functional. No styling framework.

Criterion	Excellent	Good	Satisfactory	Poor	Very Poor
Documentation and Deployment (5) Screenshots of all UIs 1 · High-level, use case and DFD diagrams 1 · References 1 · Individual contribution 1 · Challenges 1	5 All UI screenshots included and unique. Diagrams accurate and clearly labelled. References complete and consistently formatted. Every member's contribution clearly stated. Challenges discussed with genuine reflection. Source code pasted as text. Hosting and deployment steps reproducible.	4 All sections present. Diagrams mostly correct. References adequate. Contributions and challenges described briefly.	3 Some sections thin or missing. Diagrams unclear. References incomplete. Contributions stated only in general terms.	1 – 2 Several sections missing. Diagrams incorrect or absent. Few references. Contributions vague or identical for all members.	0 Report largely incomplete or missing.
Total	35 marks — awarded once for the group				

These marks are awarded once for the group, so every member receives the same mark for Table 1.

Table 2 – Individual Contribution (65 marks)

Criterion	Excellent	Good	Satisfactory	Poor	Very Poor
Web Application Features and Business Rules (18) Login and role-based access 4 · User Management 4 · Microgrid Node Management 5 · Slot Booking Management 5	15 – 18 All four function groups work through the Web API. Login verifies the user type and redirects correctly. Backoffice and Grid Operator users are created. Stations and their slots are created, updated and deleted. Reservations are created, updated and cancelled with the 7-day and 12-hour rules enforced.	12 – 14 All four function groups work with minor defects. Most business rules are enforced.	9 – 11 Two or three function groups work. Validation rules only partly enforced.	5 – 8 One function group works. Business rules are not enforced.	0 – 4 No web application functions working.
Mobile Authentication and Account Management (9) Login with role-based home 2 · Pending activation view in web app 2 · Create account in mobile 3 · Modify own account 1 · Deactivate account 1	8 – 9 Login routes each role to the correct home screen. Prosumers register with NIC as the primary key, edit their profile and request deactivation. Pending activations are visible and actionable in the web application.	6 – 7 Registration, login and profile editing work. Deactivation or the pending-activation view partly implemented.	5 Login and registration work only. No self-service edit or deactivation.	3 – 4 Login or registration present but not persisted through the API.	0 – 2 No account functions implemented.
Reservation Workflow and Booking Management (9) Create booking request 3 · Update request 2 · Cancel request 2 · Summary page after each action 2	8 – 9 Bookings created, updated and cancelled from the mobile app, with the 12-hour rule enforced on updates and cancellations and a summary page shown after each action.	6 – 7 All three operations work and a summary is shown. The 12-hour rule only partly enforced.	5 Create and one other operation work. No rule enforcement or no summary page.	3 – 4 Bookings can be created only.	0 – 2 No reservation management implemented.

Criterion	Excellent	Good	Satisfactory	Poor	Very Poor
Booking Views and Operational Dashboards (10) Current / pending bookings 2 · Booking history 2 · Filter criteria 2 · Pending reservations 2 · Count of approved future reservations 2	8 – 10 Current and pending bookings, full booking history and a working search filter, plus a dashboard showing pending reservations and the count of approved future reservations, all read live from the API.	7 Views and dashboard work with minor defects or occasional stale data.	5 – 6 About half the views work, or the dashboard shows only one of the two counts.	3 – 4 One view only, or values are hard-coded.	0 – 2 No booking views or dashboard implemented.
Grid Operator Verification and Map Features (7) Read QR code and update the job as done 2 · Show nearby stations on the map 5	6 – 7 The operator scans the prosumer QR code, verifies it against the server and finalises the job as done. Nearby grid nodes are plotted on the map from their stored latitude and longitude, with station details on selection.	5 Both work with minor defects in accuracy or detail display.	4 One of the two works; the other is missing or uses hard-coded data.	2 – 3 Scanner or map renders only, with no server verification or station data.	0 – 1 Neither implemented.
Service Integration, Local Persistence and Device Capabilities (12) Web app to Web API 2 · Mobile app to Web API 2 · SQLite local persistence 3 · Google Maps API integration 3 · QR code scanning 2	10 – 12 Both clients call the hosted Web API for every operation, with no direct database access from either client. Login details and reference data persist in SQLite on Android. Google Maps is integrated and QR codes are scanned and verified against the server.	8 – 9 Clients call the API for most operations. SQLite, Maps and QR all work with minor defects.	6 – 7 API calls work for some operations. One of SQLite, Maps or QR is missing or only partly working.	3 – 5 Clients rarely reach the API, or hold business logic and data locally. Two or more integrations missing.	0 – 2 No working integration between the clients and the web service.
Total	65 marks — awarded per student				